

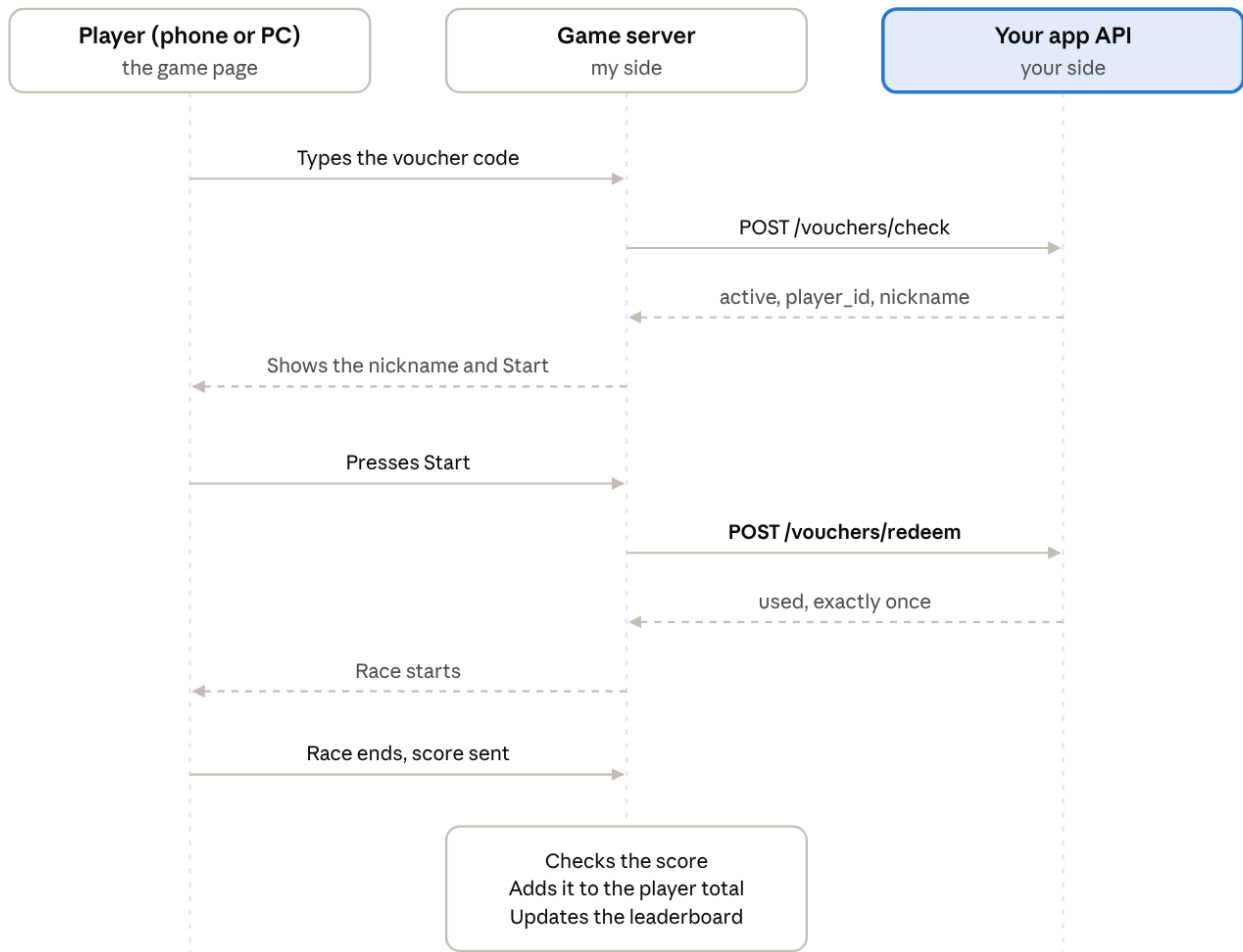
Coin Rush - Voucher API brief

Oct 4, 2026

How it works

Your app owns the vouchers and customers; my game server is the only thing that talks to it.

The browser never calls your API, the game server does



A code is checked when the player types it and used up only when the race starts, so a player who closes the page before Start keeps the voucher.

What your side builds

Your app keeps the vouchers; the game only asks it two questions. On your side:

- Voucher creation in your app, each code linked to the customer who bought it (one code = one race).
- `POST /vouchers/check` : can this code be played, and whose is it?
- `POST /vouchers/redeem` : mark the code as used when the race starts.

Everything else is on my side: the game, race sessions, score checks, the leaderboard and its database. The game does not ask for names; the nickname comes from your API.

Endpoint 1: check a voucher

POST `{base}/vouchers/check` says whether a code can be played and who owns it, without changing anything. The game server calls it when the player types the code.

Request:

```
{ "code": "7KQ4-M2XD-9PHA" }
```

Answer when the code can be played:

```
{ "ok": true, "status": "active", "voucher_id": "84211", "player_id": "u_5530", "player_name": "Rahul S" }
```

Answer when it cannot:

```
{ "ok": false, "status": "used" }
```

status	meaning
active	not used yet, can be played
used	already played
expired	past its end date
void	cancelled or refunded
not_found	no such code

Match codes without case and spaces, so `7kq4 m2xd 9pha` finds the same voucher.

Endpoint 2: redeem a voucher

POST `{base}/vouchers/redeem` marks the code as used, exactly once. The game server calls it when the player presses Start, so a code that was only checked stays playable.

Request (`session_id` is created by the game server):

```
{ "code": "7KQ4-M2XD-9PHA", "session_id": "gs_9f2c41d7" }
```

Success:

```
{ "ok": true, "status": "used", "voucher_id": "84211", "player_id": "u_5530", "player_name": "Rahul S", "used_at": "2026-10-12T14:03:22Z" }
```

- Already used by another session: `{ "ok": false, "status": "used" }`.
- Called again with the same `session_id` (a network retry): the same success answer, not an error.

- Two redeems of one code at the same moment: only one may win (see the update under Voucher codes and storage).

HTTP code	when
200	every normal answer, including ok: false
401	missing or wrong API key
429	too many requests
500	server error; the game shows "try again"

Voucher codes and storage

Codes are 12 random characters in three groups, like 7KQ4-M2XD-9PHA : far too many combinations to guess.

- Alphabet: 23456789ABCDEFGHJKLMNPQRSTUVWXYZ (no 0, O, 1, I or L, so nobody misreads a code).
- Generate with a secure random function (`random_int()` in PHP), never `rand()` or a counter.
- Unique index on the code; on the rare clash, generate again.
- One voucher = one race.

Suggested table:

column	type	note
id	int, primary key	
code	char(14), unique	uppercase, with dashes
user_id	int	the customer who owns it
status	enum: active, used, expired, void	default active
created_at	datetime	
expires_at	datetime, null	optional contest end
used_at	datetime, null	set on redeem
game_session_id	varchar(40), null	set on redeem

Redeem with one atomic update, then check that exactly one row changed:

```
UPDATE vouchers
SET status = 'used', used_at = NOW(), game_session_id = :session
WHERE code = :code AND status = 'active'
AND (expires_at IS NULL OR expires_at > NOW());
```

Player ID and nickname

Scores add up per `player_id`, so every voucher of one customer must return the same `player_id`.

- `player_id`: your user ID, or any string that never changes for that customer (max 64 characters).
- `player_name`: the nickname shown on the public leaderboard, plain text, max 20 characters. If a customer changes it, the leaderboard shows the newest one.
- No nickname set: send first name and last initial, like "Rahul S".
- Do not send email, phone or full name; the game does not need them.

Security

The browser never talks to your API; only the game server does, with a secret key.

- Header on every call: `Authorization: Bearer <API key>`. One key for testing, one for live, shared in the project chat.
- HTTPS only; refuse plain HTTP.
- Rate limit on your side, for example 60 calls per minute per key, answering 429 above that. The game server also limits each player's attempts.
- Log every redeem (code, `session_id`, time) so any prize dispute can be checked.

What I need from you

With these, I can connect the game in about a day:

- ☐ Base URL of the API, for testing and for live
- ☐ Test and live API keys
- ☐ 10 test codes: 5 for one test customer, 5 for another
- ☐ One used, one expired and one void test code
- ☐ A folder or subdomain for the game with PHP 8.1+, MySQL 5.7+ (or MariaDB 10.3+) and HTTPS, plus an empty database for the scores
- ☐ Logo (PNG or SVG, transparent background) and the business name

Before going live we test together: every status from check, redeem twice with the same and with a different `session_id`, two redeems at once, and a wrong key (401).